

Lab 3: Task-Priority kinematic control (1A)



HANDS-ON INTERVENTION: Vehicle-Manipulator Systems

Report by: PRAVIN OLI (u1999699), IFROS
(E-mail: pravin.oli.08@gmail.com)

Group : Pravin Oli and Gebreslassie Gebrecherkos Gebrezgabhier

Abstract—This lab includes redundancy resolution and task-priority control in a planar 3-link revolute manipulator. The first exercise focuses on visualizing null space motions, demonstrating how redundant degrees of freedom can be used without affecting the primary task. The second exercise implements a task-priority control algorithm, ensuring hierarchical execution of multiple tasks using null space projection.

1 INTRODUCTION

THIS lab consists of three exercises: Exercise 1: for implementation of the null space motions of a planar 3-link manipulator, with 3 revolute joints using the resolved-rate motion control algorithm and null space projection. Exercise 2: involves implementing the Task-Priority control algorithm to handle multiple tasks in a hierarchical manner, ensuring primary task execution while utilizing redundancy for secondary objectives. Exercise 3: Questions and Answers

1.1 Objectives

The objective of this lab is to study redundancy resolution in a planar 3-link manipulator. It involves implementing null space motions using resolved-rate control and developing a Task-Priority control algorithm for hierarchical task execution.

2 PLANER ROBOTIC MANIPULATOR

A planar manipulator operates in a two-dimensional plane, typically with revolute or prismatic joints.

2.1 Planar Manipulator with 3-link and 3 revolute joints.

A planar 3-link revolute manipulator consists of three rotational joints that operate in the XY-plane. The configuration of the manipulator is described by the joint angles θ_1 , θ_2 , and θ_3 .

$$\text{State} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

where:

- θ_1 is the angle of the first revolute joint.
- Erasmus Mundus Joint Masters In Intelligent Field Robotic Systems, University of Girona, Spain.

- θ_2 is the angle of the second revolute joint.
- θ_3 is the angle of the third revolute joint.

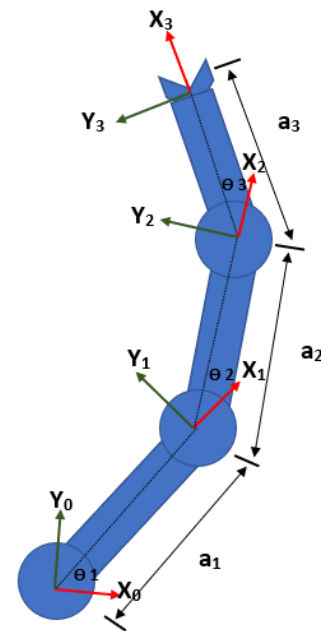


Fig. 1: 3 joint planer manipulator

i	a_i (m)	α_i (rad)	d_i (m)	θ_i (rad)
1	0.75	0	0	$q_1 = 0.2$
2	0.5	0	0	$q_2 = 0.5$
3	0.5	0	0	$q_3 = 0.3$

TABLE 1: Denavit-Hartenberg Parameters value in code

$$T_n^{n-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Redundancy and Null Space

Redundancy: A robotic manipulator is kinematically redundant if it has more degrees of freedom (DOF) than required to perform a given task. The system is redundant if:

$$n > m_i$$

where n is the total DOF and m_i is the task space dimension. Redundancy allows for increased flexibility, obstacle avoidance, and energy-efficient motion.

Null Space and Degrees of Freedom: The null space of the Jacobian J contains joint velocity commands that do not affect the execution of the task. The relationship between the range space $\mathcal{R}(J)$ and the null space $\mathcal{N}(J)$ is:

$$\dim(\mathcal{R}(J)) + \dim(\mathcal{N}(J)) = n$$

where $\dim(\mathcal{R}(J))$ represents achievable task-space velocities and $\dim(\mathcal{N}(J))$ contains internal motions.

Null Space Projector: To use the null space for secondary tasks, the null space projector is defined as:

$$P = I - J^\dagger(q)J(q)$$

where $J^\dagger$ is the pseudo-inverse of J . This ensures internal motions do not interfere with the primary task.

2.3 Task Priority

Task priority control utilizes redundancy in robotic systems to execute multiple tasks in a hierarchical manner. The approach ensures that higher-priority tasks are fulfilled first while allowing lower-priority tasks to be done in the null space of higher-priority ones.

Task Hierarchy:

- **Priority 1 (Primary Task):** Ensures the execution of the most critical task, such as the positioning of the end effector.

$$\dot{x}_1 = J_1(q)\zeta_1$$

- **Priority 2 (Secondary Task):** Utilizes the null space of the primary task to perform an additional objective, e.g., optimizing camera field of view.

$$\dot{x}_2 = J_2(q)\zeta_2$$

Combining Tasks:

- **Single Task:** The control law for a single prioritized task is given by:

$$\zeta = J_1^\dagger \dot{x}_1 + (I - J_1^\dagger J_1)y$$

where $P_1 = (I - J_1^\dagger J_1)$ is the null space projector.

- **Two Tasks:** When considering a secondary task:

$$\zeta = J_1^\dagger \dot{x}_1 + J_2^\dagger (\dot{x}_2 - J_2 J_1^\dagger \dot{x}_1) + (I - J_2^\dagger J_2)z$$

where $J_2 = J_2(I - J_1^\dagger J_1) = J_2 P_1$.

2.4 Recursive Task Priority Algorithm

The recursive formulation ensures task prioritization while maintaining consistency in null space projections. The algorithm sequentially computes the joint velocity solution while preserving higher priority tasks.

Algorithm 1 Recursive Task Priority Control

Require: List of tasks $\{J_i(q), \dot{x}_i(q)\}$, where $i \in \{1, \dots, k\}$

Ensure: Quasi-velocities $\zeta_k \in \mathbb{R}^n$

- 1: **Initialize:** $\zeta_0 \leftarrow 0^n$, $P_0 \leftarrow I^{n \times n}$
- 2: **for** $i = 1$ to k **do**
- 3: Compute projected Jacobian: $\bar{J}_i(q) \leftarrow J_i(q)P_{i-1}$
- 4: Compute velocity update:

$$\zeta_i \leftarrow \zeta_{i-1} + \bar{J}_i^\dagger(q)(\dot{x}_i(q) - J_i(q)\zeta_{i-1})$$

- 5: Update null space projector:

$$P_i \leftarrow P_{i-1} - \bar{J}_i^\dagger(q)\bar{J}_i(q)$$

- 6: **end for**

- 7: **return** ζ_k

3 IMPLEMENTATION

This lab is implemented in three python scripts: lab2 robotics.py (adopted from lab 2 Commom.py), Exercise 1.py and Exercise 2.py

3.1 lab2 robotics.py

Below is an implementation of Common.py:

```

1 import numpy as np # Import Numpy
2
3 # Denavit-Hartenberg (DH) Transformation Matrix
4 def DH(d, theta, a, alpha):
5     """
6     Function builds elementary Denavit-
7     Hartenberg transformation matrices
8     and returns the transformation matrix
9     resulting from their multiplication.
10
11     Arguments:
12     d (double): displacement along Z-axis
13     theta (double): rotation around Z-axis
14     a (double): displacement along X-axis
15     alpha (double): rotation around X-axis
16
17     Returns:
18     (Numpy array): composition of elementary DH
19     transformations
20
21     # 1. Build matrices representing elementary
22     transformations (based on input parameters).
23     T_d=np.array([[1,0,0,0],
24                  [0,1,0,0],
25                  [0,0,1,d],
26                  [0,0,0,1]])
27
28     T_theta=np.array([[np.cos(theta),-np.sin(theta),
29                      ,0,0],
30                      [np.sin(theta),np.cos(theta),
31                      ,0,0],
32                      [0,0,1,0],
33                      [0,0,0,1]])
34
35     T_a=np.array([[1,0,0,a],
36                  [0,1,0,0],
37                  [0,0,1,0],
38                  [0,0,0,1]])

```

```

32         [0,0,0,1]])
33
34     T_alpha=np.array([[1,0,0,0],
35                       [0,np.cos(alpha),-np.sin(alpha),0],
36                       [0,np.sin(alpha),np.cos(alpha),0],
37                       [0,0,0,1]])
38
39
40     # 2. Multiply matrices in the correct order (
41     result in T).
42     T=T_d @ T_theta @ T_a @ T_alpha
43     return T
44
45 # Forward Kinematics Computation
46 def kinematics(d, theta, a, alpha):
47     '''
48     Functions builds a list of transformation
49     matrices, for a kinematic chain,
50     described by a given set of Denavit-
51     Hartenberg parameters.
52     All transformations are computed from the
53     base frame.
54
55     Arguments:
56     d (list of double): list of displacements
57     along Z-axis
58     theta (list of double): list of rotations
59     around Z-axis
60     a (list of double): list of displacements
61     along X-axis
62     alpha (list of double): list of rotations
63     around X-axis
64
65     Returns:
66     (list of Numpy array): list of
67     transformations along the kinematic
68     chain (from the base frame)
69
70     '''
71
72     #Initialize the base transformation matrix (
73     Identity matrix)
74     T = [np.eye(4)]
75
76     # For each set of DH parameters:
77     # 1. Compute the DH transformation matrix.
78     # 2. Compute the resulting accumulated
79     transformation from the base frame.
80     # 3. Append the computed transformation to T.
81
82     for i in range(len(d)):
83         Ti = DH(d[i], theta[i], a[i], alpha[i]) #
84         Compute DH transformation
85         T.append(T[-1] @ Ti) # Compute cumulative
86         transformation
87     return T
88
89 # Compute Jacobian Matrix, Inverse kinematics
90 def jacobian(T, revolute):
91     '''
92     Function builds a Jacobian for the end-
93     effector of a robot,
94     described by a list of kinematic
95     transformations and a list of joint
96     types.
97
98     Arguments:
99     T (list of Numpy array): list of
100     transformations along the kinematic
101     chain of the robot (from the base frame)
102     revolute (list of Bool): list of flags
103     specifying if the corresponding joint is
104     a revolute joint
105
106     Returns:
107     (Numpy array): end-effector Jacobian
108     '''
109     # 1. Initialize J and 0.
110     # 2. For each joint of the robot
111     # a. Extract z and o.
112     # b. Check joint type.
113     # c. Modify corresponding column of J.
114
115     # 1. Initialize J and 0.
116     J = np.zeros((6,len(T)-1))
117
118     # 2. For each joint of the robot
119     for i in range (len(T)-1):
120         # a. Extract z and o.
121         if i == 0:
122             z = np.array([0,0,1]) #Base Z-axis
123         else:
124             z = T[i][0:3,2] # Extract the Z-axis
125             from the transformation matrix
126
127         o = T[-1][0:3, 3] - T[i][0:3, 3] # Compute
128         position vector
129
130         zero = np.zeros(3)
131
132         # b. Check joint type.
133         if revolute[i]:
134             # c. Modify corresponding column of J.
135             J[:,i] = np.concatenate((np.cross(z, o),
136                                     z)).reshape((6,))
137         else:
138             J[:,i] = np.concatenate((z,zero)).
139             reshape((6,))
140     return J
141
142 # Damped Least-Squares Inverse of Jacobian
143 def DLS(J, damping):
144     '''
145     Function computes the damped least-squares (
146     DLS) solution to the matrix inverse
147     problem.
148
149     Arguments:
150     A (Numpy array): matrix to be inverted
151     damping (double): damping factor
152
153     Returns:
154     (Numpy array): inversion of the input matrix
155     '''
156     # Zeta = J( )T (J( )*J( )T + **2 *
157     I) 1 xE
158     DLS = J.T @ np.linalg.inv(J @ J.T + damping**2 *
159                               np.eye(J.shape[0]))
160
161     return DLS # Implement the formula to compute
162     the DLS of matrix A.
163
164 #Jacobian Inverse J_Inv
165 def PseudoInv(J):
166     '''
167     Function computes the pseudo-inverse of the
168     Jacobian matrix.
169
170     Arguments:
171     J (Numpy array): Jacobian matrix
172
173     Returns:
174     (Numpy array): Pseudo-inverse of the input
175     matrix
176     '''
177     return np.linalg.pinv(J)
178
179 # Extract characteristic points of a robot projected
180 on X-Y plane
181 def robotPoints2D(T):

```

```

151 '''
152     Function extracts the characteristic points
153     of a kinematic chain on a 2D plane,
154     based on the list of transformations that
155     describe it.
156     Arguments:
157     T (list of Numpy array): list of
158     transformations along the kinematic
159     chain of the robot (from the base frame)
160     Returns:
161     (Numpy array): an array of 2D points
162 '''
163 P = np.zeros((2,len(T))) # Create an empty array
164 for 2D coordinates
165 for i in range(len(T)):
166     P[:,i] = T[i][0:2,3] # Extract X and Y
167     positions
168 return P

```

3.2 Exercise 1.py

Below is an implementation of Exercise 1.py:

```

1 # Import necessary libraries
2 from lab2_robotics import * # Includes numpy import
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as anim
5
6 # Robot definition (3 revolute joint planar
7   manipulator)
8 d = np.zeros(3) #
9 displacement along Z-axis
10 q = np.array([0.2, 0.5, 0.3]).reshape(3,1) #
11 rotation around Z-axis (theta)
12 alpha = np.zeros(3) #
13 rotation around X-axis
14 a = np.array([0.75, 0.5, 0.5]) #
15 displacement along X-axis (link length)
16 revolute =[True, True, True] # flags
17 specifying the type of joints
18 k = np.diag([1, 1]) #Gain
19
20 # Setting desired position of end-effector to the
21   current one
22 T = kinematics(d, q.flatten(), a, alpha) # flatten()
23   needed if q defined as column vector !
24 sigma_d = T[-1][0:2,3].reshape(2,1)
25
26 # Simulation params
27 dt = 1.0/60.0
28 Tt = 10 # Total simulation time
29 tt = np.arange(0, Tt, dt) # Simulation time vector
30
31 # Drawing preparation
32 fig = plt.figure()
33 ax = fig.add_subplot(111, autoscale_on=False, xlim
34   =(-2, 2), ylim=(-2,2))
35 ax.set_title('Simulation')
36 ax.set_aspect('equal')
37 ax.set_xlabel('x[m]')
38 ax.set_ylabel('y[m]')
39 ax.grid()
40 line, = ax.plot([], [], 'o-', lw=2) # Robot
41   structure
42 path, = ax.plot([], [], 'c-', lw=1) # End-effector
43   path
44 point, = ax.plot([], [], 'rx') # Target
45 PPx = []
46 PPy = []
47
48 q1 = []
49 q2 = []
50 q3 = []
51 timestamp = []

```

```

41 # Simulation initialization
42 def init():
43     line.set_data([], [])
44     path.set_data([], [])
45     point.set_data([], [])
46     return line, path, point
47
48 # Simulation loop
49 def simulate(t):
50     global q, a, d, alpha, revolute, sigma_d
51     global PPx, PPy
52
53     # Update robot
54     T = kinematics(d, q.flatten(), a, alpha)
55     J = jacobian(T, revolute)
56
57     # Update control
58     sigma = T[-1][0:2, -1].reshape(2,1) # Current
59     position of the end-effector
60     err = sigma_d - sigma # Error
61     in position
62     Jbar =J[:2, :]
63     J_Inv = PseudoInv(Jbar) # Task
64     Jacobian
65     P = np.eye(3) - (J_Inv @ Jbar) # Null
66     space projector
67     y = np.array([0.5*np.sin(t), np.cos(t), 1.5*np.
68     sin(t)]).reshape(3,1) # Arbitrary joint
69     velocity
70     dq = (J_Inv @ (k @ err)) + (P @ y)
71     # Control signal
72     q = q + dt * dq # Simulation update
73
74     # Update drawing
75     PP = robotPoints2D(T)
76     line.set_data(PP[0,:], PP[1,:])
77     PPx.append(PP[0,-1])
78     PPy.append(PP[1,-1])
79     path.set_data(PPx, PPy)
80     point.set_data(sigma_d[0], sigma_d[1])
81
82     q1.append(q[0])
83     q2.append(q[1])
84     q3.append(q[2])
85     timestamp.append(t)
86
87     return line, path, point
88
89 # Run simulation
90 animation = anim.FuncAnimation(fig, simulate, np.
91   arange(0, 60, dt),
92   interval=10, blit=
93     True, init_func=
94     init, repeat=
95     False)
96
97 plt.show()
98
99 # Drawing The Position of each joint with respect
100   time
101 fig_joint = plt.figure()
102 ax = fig_joint.add_subplot(111, autoscale_on=False,
103   xlim=(0, 60), ylim=(-2, 3))
104 ax.set_title("joint positions")
105 ax.set_xlabel("Time[s]")
106 ax.set_ylabel("Angle[rad]")
107 ax.grid()
108 plt.plot(timestamp, q1, label="q1 :0.5*sin curve")
109 plt.plot(timestamp, q2, label="q2 : cos curve")
110 plt.plot(timestamp, q3, label="q3 : 1.5*sin curve")
111 ax.legend()
112 plt.show()

```

3.3 Exercise 2.py

Below is an implementation of Exercise 2.py:

```

67 # Import necessary libraries
68 from lab2_robotics import * # Includes numpy import
69 import matplotlib.pyplot as plt
70 import matplotlib.animation as anim
71
72 # Robot definition (3 revolute joint planar
73 manipulator)
74 d = np.zeros(3) #
75 displacement along Z-axis
76 q = np.array([0.2, 0.5, 0.3]).reshape(3,1) #
77 rotation around Z-axis (theta)
78 alpha = np.zeros(3) #
79 rotation around X-axis
80 a = np.array([0.75, 0.5, 0.5]) #
81 displacement along X-axis (link length)
82 revolute =[True, True, True] # flags
83 specifying the type of joints
84
85 # Desired values of task variables
86 sigma1_d = np.array([0.0, 1.0]).reshape(2,1) #
87 Position of the end-effector
88 sigma2_d = np.array([[0.0]]) # Position of joint 1
89
90 #gains
91 K1 = np.diag([1, 1])
92 K2 = np.array([1])
93 max_val = np.array([1,0.8,0.8])
94
95 # Simulation params
96 dt = 1.0/60.0
97 Tt = 10 # Total simulation time
98 tt = np.arange(0, Tt, dt) # Simulation time vector
99
100 # Drawing preparation
101 fig = plt.figure()
102 ax = fig.add_subplot(111, autoscale_on=False, xlim
103 =(-2, 2), ylim=(-2,2))
104 ax.set_title('Simulation')
105 ax.set_aspect('equal')
106 ax.set_xlabel('x[m]')
107 ax.set_ylabel('y[m]')
108 ax.grid()
109 line, = ax.plot([], [], 'o-', lw=2) # Robot
110 structure
111 path, = ax.plot([], [], 'c-', lw=1) # End-effector
112 path
113 point, = ax.plot([], [], 'rx') # Target
114 PPx = []
115 PPy = []
116
117 e1 = []
118 e2 = []
119
120 timestamp = []
121 prev_time = 0
122
123 # Task priority uncomment the one you want to use.
124 #t_priority = "End_Effector"
125 t_priority = "JOINT 1 position"
126
127 # Simulation initialization
128 def init():
129     global sigma1_d, prev_time
130     line.set_data([], [])
131     path.set_data([], [])
132     point.set_data([], [])
133     prev_time = timestamp[-1] if timestamp else 0
134     sigma1_d = np.random.uniform(-1, 1, size=(2, 1))
135     return line, path, point
136
137 # Simulation loop
138 def simulate(t):
139     global q, a, d, alpha, revolute, sigma1_d,
140     sigma2_d
141     global PPx, PPy, prev_time, max_val
142
143     # Update robot
144     T = kinematics(d, q.flatten(), a, alpha)
145     J = jacobian(T, revolute)
146
147     # Update control
148     if t_priority == "End_Effector":
149         # TASK 1
150         sigma1 = T[-1][0:2, -1].reshape(2, 1) #
151         Current position of the end-effector
152         err1 = sigma1_d - sigma1 #
153         Error in Cartesian position
154         J1 = J[:2, :]
155         J_Inv = DLS(J1, 0.1) #
156         Jacobian of the first task
157         P1 = np.eye(3) - (PseudoInv(J1) @ J1)
158         # Null space projector
159
160         # TASK 2
161         sigma2 = q[0] # Current
162         position of joint 1
163         err2 = sigma2_d - sigma2 # Error in
164         joint position
165         J2 = np.array([1, 0, 0]).reshape(1, 3) #
166         Jacobian of the second task
167         J2bar = J2 @ P1 # Augmented
168         Jacobian
169
170         # Combining tasks
171         dq1 = (J_Inv @ K1 @ err1).reshape(3, 1)
172         # Velocity for the first
173         task
174         dq12 = dq1 + (DLS(J2bar, 0.1) @ (err2 - J2 @
175         dq1)) # Velocity for both tasks
176
177     elif t_priority == "JOINT 1 position":
178         # TASK 1
179         sigma2 = q[0] # Current
180         position of the end-effector
181         err2 = sigma2_d - sigma2 # Error in
182         Cartesian position
183         J1 = np.array([1, 0, 0]).reshape(1, 3)
184         J_Inv = DLS(J1, 0.1) #
185         Jacobian of the first task
186         P1 = np.eye(3) - (PseudoInv(J1) @ J1) #
187         Null space projectorJinv # Null space
188         projector
189
190         # TASK 2
191         sigma1 = T[-1][0:2, -1].reshape(2, 1) #
192         Current position of joint 1
193         err1 = sigma1_d - sigma1 #
194         Error in joint position
195         J2 = J[:2, :] # Jacobian
196         of the second task
197         J2bar = J2 @ P1 # Augmented
198         Jacobian
199
200         # Combining tasks
201         dq1 = (J_Inv @ err2).reshape(3, 1)
202         # Velocity for the first
203         task
204         dq12 = dq1 + (DLS(J2bar, 0.1)) @ (err1 - J2 @
205         dq1) # Velocity for both tasks
206
207     # Apply velocity scaling to limit max joint
208     velocity
209     s = np.max(np.abs(dq12) / max_val) # Compute
210     scaling factor
211     dq12 = np.where(s > 1, dq12 / s, dq12) # Scale
212     velocities
213
214     q = q + dq12 * dt # Simulation update
215
216     # Update drawing
217     PP = robotPoints2D(T)
218     line.set_data(PP[0,:], PP[1,:])
219     PPx.append(PP[0,-1])

```

```

118 PPy.append(PP[1,-1])
119 path.set_data(PPx, PPy)
120 point.set_data(sigma1_d[0], sigma1_d[1])
121
122 norm_err1= np.linalg.norm(err1)
123 norm_err2= np.linalg.norm(err2)
124
125 e1.append(norm_err1)
126 e2.append(norm_err2)
127
128 timestamp.append(t + prev_time)
129
130 return line, path, point
131
132 # Run simulation
133 animation = anim.FuncAnimation(fig, simulate, np.
134                               arange(0, 10, dt),
135                               interval=10, blit=
136                               True, init_func=
137                               init, repeat=
138                               True)
139
140 plt.show()
141
142 # post simulation plotting
143 fig_joint = plt.figure()
144 ax = fig_joint.add_subplot(111, autoscale_on=False,
145                             xlim=(0, 60), ylim=(-1, 2))
146 ax.set_title("Task-Priority (two tasks)")
147 ax.set_xlabel("Time[s]")
148 ax.set_ylabel("Error")
149 ax.grid()
150 plt.plot(timestamp, e1, label="e1 (end-effector
151         position)")
152 plt.plot(timestamp, e2, label="e2 (joint 1 position)
153         ")
154 ax.legend(loc='upper right')
155 plt.show()

```

4 RESULTS AND DISCUSSION

The result of lab is obtained as:

4.1 Exercise 1 (visualize the null space motions)

End-Effector Motion Simulation

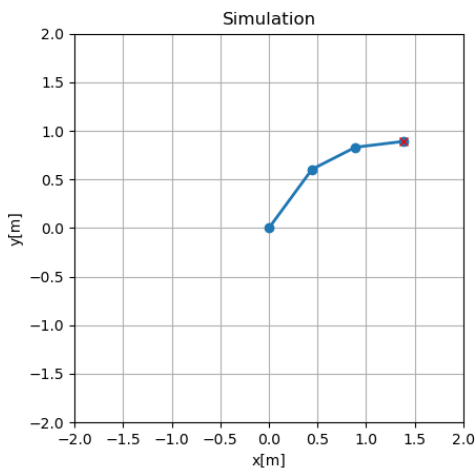


Fig. 2: Simulation of the manipulator, including end-effector goal.

- The Fig.2: visualizes the movement of a **3-link planar manipulator** in a 2D space.

- Null space motion** is utilized, ensuring that the robot follows the primary task while also accommodating internal joint motions.

Joint Position Evolution Over Time

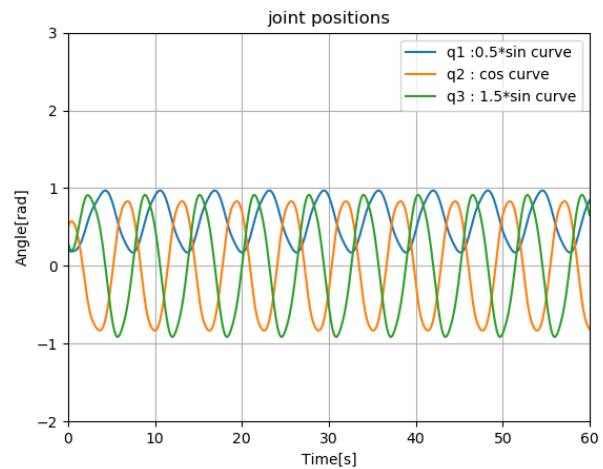


Fig. 3: Positions of robot's joints during simulation.

- The Fig.3: shows the time evolution of the three **joint angles** (q_1, q_2, q_3) over a **60-s simulation**.
- Each joint follows a predefined function:
 - q_1 follows a $0.5 \sin(t)$ curve.
 - q_2 follows a $\cos(t)$ curve.
 - q_3 follows a $1.5 \sin(t)$ curve.
- These oscillations represent the robot's internal movements in the **null space**, ensuring that **secondary tasks** (such as preferred joint configurations) can be executed without disturbing the primary motion.

4.2 Exercise 2 (Task-Priority control)

4.2.1 End-Effector Priority

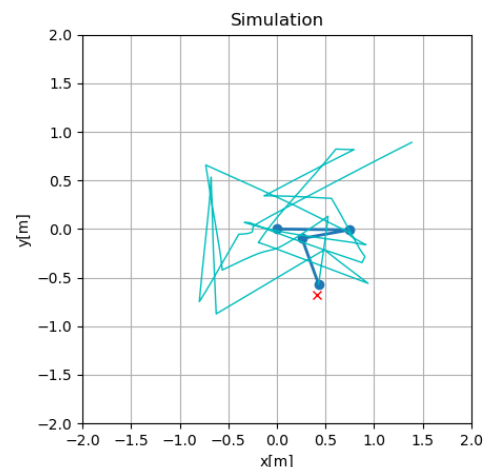


Fig. 4: Simulation for end-effector priority

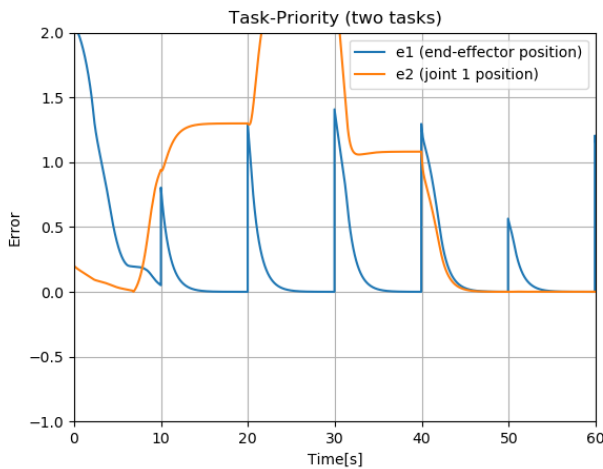


Fig. 5: Evolution of control error for End-Effector Priority

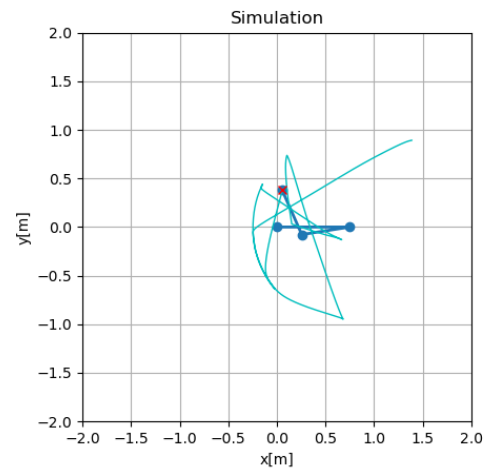


Fig. 6: Simulation for Joint 1 Priority

- The Fig.4: shows the **end-effector trajectory** of a 3-link planar manipulator attempting to reach a target position.
- The Fig.5: shows the **error evolution** over time for two tasks:
 - e_1 (blue) represents the error in the **end-effector position**.
 - e_2 (orange) represents the error in **joint 1 position**.
- The error **gradually decreases**, showing that the robot is achieving the tasks over time.
- **Repeated spikes** in error indicate changes in task priority or sudden updates in target positions.
- The simulation uses **task-priority control** to execute multiple tasks hierarchically.
- The error plot confirms that the primary task (end-effector position) is prioritized before the secondary task (joint position).

4.2.2 Joint 1 Priority

- The Fig.6: shows the **end-effector trajectory** when **joint 1 position** is prioritized over the end-effector task. The manipulator moves towards the target, but its motion is **constrained** by the higher-priority joint position task.
- Compared to the previous case, the **end-effector path is less erratic**, but it does not directly reach the desired position due to the imposed joint constraints.
- The Fig.7: represents the **error evolution** for two tasks:
 - e_1 (blue) is the error in the **end-effector position**.
 - e_2 (orange) is the error in **joint 1 position**.
- The error in **joint 1 position** (e_2) remains very small, confirming that it is successfully controlled as the highest priority task.
- The error in the **end-effector position** (e_1) reduces over time but exhibits **disruptions** due to the strict prioritization of joint 1.

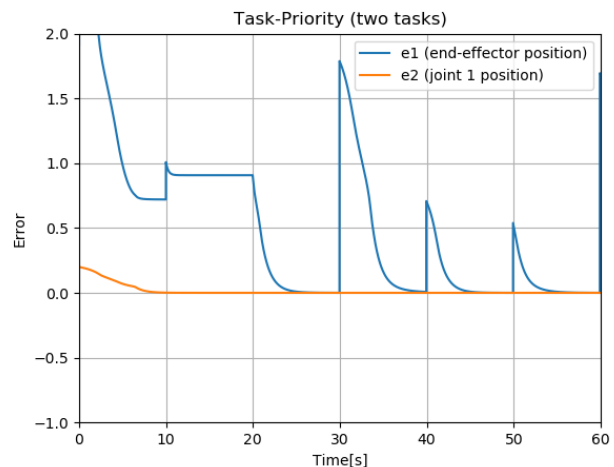


Fig. 7: Evolution of control error for Joint 1 Priority

- The periodic spikes indicate **resets or changes in the control parameters**, due to variations in the target position.

5 QUESTIONS

1. What are the advantages and disadvantages of redundant robotic systems?

Advantages:

- **Increased Dexterity:** Redundancy allows robots to perform more complex and flexible motions, improving maneuverability in constrained environments.
- **Obstacle Avoidance:** The additional degrees of freedom enable the robot to avoid obstacles while still achieving the primary task.
- **Fault Tolerance:** If a joint fails, the robot can still complete the task by reconfiguring its remaining joints.
- **Secondary Task Execution:** The null space motion can be used to perform additional tasks, such as avoiding singularities, optimizing posture, or minimizing joint torques.

Disadvantages:

- **Increased Computational Complexity:** Redundancy requires solving underdetermined systems, leading to higher computational costs.
- **Control Challenges:** Managing multiple tasks and resolving redundancy in real time can be complex and requires advanced control algorithms.
- **Higher Cost and Maintenance:** More joints mean increased hardware costs, higher maintenance requirements, and potential reliability issues.
- **Stability Issues:** While redundancy helps avoid singularities, improper redundancy resolution can introduce instability in motion.
- **Longer Planning Time:** Finding an optimal configuration in redundant robots can take longer, especially in dynamic environments.

2. What is the meaning and practical use of a weighting matrix W , that can be introduced in the pseudo inverse/DLS implementation?

Meaning: A weighting matrix W is introduced in the pseudo-inverse or damped least squares (DLS) method to modify the influence of different joint velocities in the inverse kinematics solution. It is used in the weighted Moore-Penrose inverse:

$$J_W^\dagger = W^{-1} J^T (J W^{-1} J^T)^{-1}$$

where W is a positive definite diagonal matrix that assigns different importance to each joint.

Practical Use:

- **Joint Limit Avoidance:** Assigning higher weights to joints near their limits discourages excessive motion in those joints.
- **Precision Improvement:** Assigning lower weights to more precise joints allows them to contribute more to the motion.
- **Obstacle Avoidance:** A weighting matrix can be dynamically adjusted to prevent collisions by penalizing motion in risky directions.

6 CONCLUSION

In this lab, redundancy resolution using null space projection and task-priority control in a planar 3-link manipulator was implemented. The results confirm that null space motions allow for secondary task execution without disturbing the primary task, and task-priority control ensures that higher-priority objectives are satisfied first. When the end-effector position is prioritized, joint motion adapts accordingly. Conversely, when joint 1 is prioritized, the end-effector trajectory is constrained by the joint's movement. The error analysis validates the effectiveness of hierarchical task execution.